BSc (Hons) in **Information Technology Specialized in AI**

Faculty of Computing

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Practical 01

> **Objective & Lecture Mapping:** Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the Git Hub Repo. Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance Measure: to define what success looks like for the agent
Environment: To define the world in which the agent operates
Actuators: To enable the agent to affect to the environment and to define what the

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

self.walls
self.food
self.agent_pos

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

This has multi-agent baseline environment. Because it initialized as num_opponents = 2, therefore this should be multi-agent.

Task 1.2: The Perception Subsystem (get_percept)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete history of everything an agent has perceived through it sensors, each successive time step, showing how its observations change as it interacts with the environment.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

get_percept() represents the Sensors component of PEAS.
Because it is the function that gathers everything that the agent can perceive from the environment.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

Partially Observable
The agent can see its immediate surroundings but not the full grid state. Food locations are hidden until the agent steps on them. The agent must explore to

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Performance Measure is updated.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

It is structurally important to evaluate external environment changes rather than internal processing effort because the agent's ultimate purpose is to achieve real-world goals, not computational busywork. If we rewarded internal effort, the agent

Part 2: Guided Modification & Deep Theory Mapping

> Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (__init__)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?**

If the trap exists in the environment but the agent has no sensor for them, the environment becomes partially observable instead of fully observable. The agent can't know everything it needs from its percepts alone.

Step 2.2: Updating the Perception Subsystem (get_percept)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

By adding this to the agent, it will detect danger before stepping into a trap, instead of only finding out after being hit. This supports rationality because the agent can compare nearby moves in advance and avoid toxic tiles, maximizing its

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

internal actions rather than the external state of the environment lets a mathematically optimal agent exploit the performance measure by repeatedly performing the rewarded action without achieving the intended goal. Consequently, the agent maximizes its score while the environment remains dirty,

Finalize and Lock Lab 01 Analytical Evaluation

Lab 01 code analysis and theoretical evaluation complete and verified!

FACULTY OF COMPUTING